

A fast, from-scratch search for proto-essential subgroups in small p -groups, with a complete determination for symplectic-type groups up to order 5^7 and 2^{10}

October 7, 2026

Abstract

We give a pure-Python, from-scratch implementation of a fast search for the candidates for *essential* subgroups in the sense of the Alperin–Goldschmidt fusion theorem, following the reduction of Gautam (Section 3 of [1]) to centralizers of order- p elements in a refined central series, rather than the full subgroup lattice used by the original Parker–Semeraro algorithm [2]. The implementation consists of a compiled power-commutator (PC) group engine, explicit verified automorphisms of extraspecial p -groups via Winter’s theorem, and the candidate/filter/orbit pipeline of Gautam’s Theorem 3.6. We apply it to extraspecial p -groups of order up to $5^7 = 78125$ (a case outside the range of the small-group-library-based computations of [2]) and to 2-groups up to order $2^{10} = 1024$. We prove, and the computation confirms at every size tested, that extraspecial p -groups of rank $n \geq 2$ have *no* proto-essential subgroups at all – a short, general structural fact about class-2 p -groups with derived subgroup of order p , worked out here because the pipeline needed to be certain it was not a bug. We document the one serious scaling bottleneck we found (an $O(|E|)$ -per-candidate cost in building generating sets that becomes $O(|E|^2)$ -like in aggregate) and its fix, with a direct before/after timing comparison at order 5^7 .

Contents

1	Introduction	1
1.1	Honest scope	2
2	A compiled power-commutator group engine	2
2.1	Collection from the left	2
2.2	Everything else is table lookups	3
2.3	Correctness, not just speed	3
3	Explicit automorphisms via Winter’s theorem	3
4	The proto-essential search pipeline	4
4.1	Candidate generation	4
4.2	Cheap, $\text{Aut}(E)$ -free filters	5
4.3	Deduplication and the $\text{Aut}(S)$ -orbit reduction	5
4.4	The radical test	5
5	No nonabelian radical subgroups in a class-2 group with $S' = p$	6
5.1	What zero essential subgroups means for the fusion system	7

6	Results	7
6.1	Validation	7
6.2	Small cases	7
6.3	The 5^7 and 2^{10} showcase	8
7	A scaling bottleneck, found and fixed	8
8	Discussion and future work	9

1 Introduction

Let S be a finite p -group. A *saturated fusion system* $\mathcal{F}$ on S records compatible families of S -conjugation-like maps between subgroups of S that could arise from S being a Sylow p -subgroup of some finite group. The *Alperin–Goldschmidt fusion theorem* says that $\mathcal{F}$ is entirely determined by $\text{Aut}_{\mathcal{F}}(S)$ together with $\text{Aut}_{\mathcal{F}}(E)$ for E ranging over the (finitely many, up to S -conjugacy) $\mathcal{F}$ -*essential* subgroups: S -centric, fully $\mathcal{F}$ -normalized subgroups E for which $\text{Out}_{\mathcal{F}}(E)$ has a strongly p -embedded subgroup. Classifying the saturated fusion systems on a given S therefore reduces, in large part, to finding the essential subgroups – or at least a manageable list of *candidates* for them, on which a combinatorial search over compatible automorphism assignments can then be run.

The original Parker–Semeraro approach [2] takes every S -centric subgroup of S as a candidate. This requires enumerating the subgroup lattice of S (or of $S/Z(S)$), which is already infeasible for groups of moderate order: the companion MAGMA work referenced in this project’s top-level `README.md` records that $S = C_3 \wr C_3 \wr C_3$, of order 3^{13} , defeats the naive approach outright (it exhausts a 7 GB memory limit in GAP), while even S of order p^6 is already slow.

Gautam’s Theorem 3.6 ([1], Section 3) replaces this with a dramatically smaller candidate set: fix a central series $S = L_0 > L_1 > \cdots > L_n = 1$ refining the lower central series, with every factor of order p . Every candidate is then S -conjugate to a centralizer $C_S(xA/A)$ for A one of the L_i *properly* contained in $S' = [S, S]$, and xA an element of order p in the (much smaller) quotient S/A . The candidates come from conjugacy classes of elements in a handful of small quotient groups, never from the subgroup lattice of S itself.

This paper documents a complete, from-scratch Python implementation of this pipeline – a compiled PC-group engine, explicit verified automorphisms, and Gautam’s candidate/filter/orbit reduction – together with:

- a cross-validation against GAP at small orders (Section 6.1),
- a proof that extraspecial p -groups of rank $n \geq 2$ have no proto-essential subgroups at all (Section 5), a fact we needed to establish rigorously once the computation kept returning zero and we had to rule out a bug,
- showcase computations on extraspecial groups up to order $5^7 = 78125$ and 2-groups up to order $2^{10} = 1024$ (Section 6), and
- a concrete record of the one scaling bottleneck we hit while pushing from order p^5 to p^7 , and its fix (Section 7).

All code is in the `fastfusion/` package of this repository; this paper does not restate the code but explains the design decisions and proves the one genuinely new mathematical claim (Theorem 5.2) that the implementation relies on and that the empirical results confirm.

1.1 Honest scope

This implementation covers the *proto-essential subgroup search* – the expensive, previously infeasible stage. It does not re-implement, in Python, the subsequent combinatorial search over automizer assignments and the saturation test (the lazy, fusion-graph-free engine already written in GAP/-MAGMA elsewhere in this repository for $C_3 \wr C_3 \wr C_3$, which does have nonzero essential subgroups and so genuinely needs that stage). For every group in our showcase (Section 6) the proto-essential search returns the empty list, and Section 5.1 explains why that already determines *every* saturated fusion system on these groups without any further search being necessary. Extending the Python engine with the general automizer-assignment search is future work, useful for groups (such as $C_3 \wr C_3 \wr C_3$) that do have nonzero essential subgroups; it is not needed for the results reported here.

2 A compiled power-commutator group engine

Every group S in this project is given by a *power-commutator (PC) presentation*: generators $x_1, \dots, x_n$, all of the same prime relative order p , with

$$x_i^p = w_i, \quad [x_i, x_j] = c_{ij} \quad (i < j),$$

where every word w_i and c_{ij} involves only generators of index *strictly greater* than i (resp. j). Equivalently, $L_i = \langle x_i, \dots, x_n \rangle$ is a central series of S refining the lower central series with elementary abelian factors of order p – precisely the series Theorem 3.6 needs, built in by the choice of presentation rather than computed separately.

2.1 Collection from the left

The raw presentation (PCPresentation in `pcgroup.py`) comes with `collect`, a general “collection from the left” algorithm [4, Ch. 9]: repeatedly swap an out-of-order adjacent pair $x_a x_b$ ($a > b$) into $x_b x_a [x_a, x_b]$, splicing in the commutator word, then collapse any run of $\geq p$ copies of one generator using its power relation. Both substitutions only ever introduce generators strictly later than the ones involved, so the process terminates, for a PC presentation of *any* nilpotency class – no assumption of class 2 is built into the engine itself (only our showcase groups happen to have class 2). `collect` is only ever used to *compile* the presentation: a single breadth-first pass over the Cayley graph (right-multiplication by each generator) builds every element’s normal-form exponent vector once.

2.2 Everything else is table lookups

After compilation, `CompiledPGroup` represents S on indices $0, \dots, p^n - 1$ via a single table `right[g,i]` = index of `elements[i] · xg+1`. Every other operation is derived from this table alone, and `collect` is never called again:

- **Multiplication** $i \cdot j$: decompose `elements[j]` into its normal-form letters and right-multiply from i , one letter at a time – $O(n)$ table lookups.
- **Inversion**: Lagrange’s theorem, $x^{-1} = x^{|S|-1}$, via binary exponentiation on the multiplication just defined – $O(\log |S|)$ multiplications, and notably *no* need to know the actual order of x (which can exceed p even though every relative order in the presentation is p).
- **Conjugation, commutators, homomorphism application**: built from multiplication and inversion in the same way.

- **Centralizers and normalizers:** for $h \in S$, the *conjugation vector* $\text{cv}[i] = \text{index of } h \text{ conjugated by } \text{elements}[i]$, for every i , is computed in a single $O(|S|)$ pass by reusing the BFS parent tree from compilation (conjugation by a product $y \cdot x_g$ is conjugation by y then by x_g , and the table $\text{conj}[g, \cdot]$ of conjugation by each single generator is precomputed once). Centralizing or normalizing a generating set is then a handful of vectorized NumPy boolean-mask operations over $0, \dots, p^n - 1$.

At the sizes used here ($p^n \leq 5^7 = 78125$ or $2^{10} = 1024$) every table fits comfortably in memory, and every one of the operations above costs at most $O(n \cdot p^n)$ elementary array operations – which is what makes the engine fast enough to run Gautam’s reduction at these sizes at all.

2.3 Correctness, not just speed

Three things are checked independently of “it ran without crashing”: `verify_endomorphism` re-checks a proposed endomorphism against *every* defining relation of the presentation (necessary and sufficient for a map on generators to extend to a homomorphism); `is_automorphism` adds a surjectivity check via `closure`; and `tests/gap_oracle.py` independently builds the identical presentation in GAP, using the `polycyclic` package’s `FromTheLeftCollector` (which matches our PCGS exactly, rather than going through a coset-enumeration-based route from a free presentation, which proved unreliable for groups of this size), and compares order, center, derived subgroup, Frattini subgroup, nilpotency class, exponent, number of conjugacy classes, element-order profile and, where available, the small-group library ID. Every presentation used in the showcase passes this cross-check (Section 6.1).

3 Explicit automorphisms via Winter’s theorem

Gautam’s $\text{Aut}(S)$ -orbit reduction (Section 4) needs a generating set for (a large enough subgroup of) $\text{Aut}(S)$. For S extraspecial of order p^{1+2n} , with $Z(S) = \Phi(S) = S' = \langle z \rangle$ of order p and $V = S/Z(S)$ carrying the nondegenerate alternating commutator form B , Winter’s theorem [3] gives

$$\text{Aut}(S) = \text{Inn}(S) \cdot \text{Out}(S), \quad \text{Out}(S) \cong \begin{cases} \text{CSp}(2n, p) & p \text{ odd (exponent-}p \text{ type)}, \\ O^\varepsilon(2n, 2) & p = 2, \end{cases}$$

where $\varepsilon = \pm$ is the Witt type of the quadratic refinement q of B that records the p -th power map ($x_i^2 = z^{q(x_i)}$, etc.) for $p = 2$.

Rather than hand-deriving a verified minimal generating set for these classical groups symbolically – easy to get a sign or isotropy condition subtly wrong, especially in characteristic 2, as we found out directly (Section 3) – `autgroup.py` takes a *generate-and-verify* approach. For odd p , `winter_generators_odd_p` writes down the standard generators of $\text{SL}(2, p)$ on each hyperbolic plane (swap-invert, shears, diagonal scalars), a global scalar, and adjacent-plane swaps and mixing transvections; for $p = 2$, `winter_generators_p2` instead searches systematically (every pairwise coordinate swap, and for every basis position and every small subset of the others, the shear of adding that subset onto the position). Every candidate is lifted to a map on all PC generators and kept only if `CompiledPGroup.is_automorphism` accepts it – so every automorphism used anywhere in this project is verified computationally against the defining relations, never merely trusted.

A genuine bug this approach caught. An earlier, hand-derived version of the $p = 2$ orthogonal transvection formula preserved the bilinear form B but not the quadratic refinement q itself: direct

computation showed $q(T_v(w)) = q(w) + c \cdot (q(v) + 1)$ when $q(v) = 0$, not $q(w)$ as the formula assumed. The systematic search in `winter_generators_p2` replaced it entirely, and is independently checked in `tests/test_autgroup.py` against the exact order of $\text{Aut}(S)$ for $n = 1$ ($|\text{Aut}(D_8)| = 8$, $|\text{Aut}(Q_8)| = 24$) and against transitivity on the natural candidate set (the 1-dimensional subspaces of V) for $n = 2$.

A fast surjectivity test. Checking that a candidate map is an automorphism still needs a surjectivity test, and `is_automorphism`'s generic version (`closure`, an $O(|S|)$ flood) is not the bottleneck at these sizes but is also not free when thousands of candidates are tried. By the Burnside basis theorem, a set of elements generates a p -group iff their images span the Frattini quotient; for extraspecial S , $\Phi(S) = Z(S)$, so $S/\Phi(S) = V = \mathbb{F}_p^{2n}$, and `is_automorphism_fast` in `autgroup.py` replaces the $O(|S|)$ closure with a $2n \times 2n$ invertibility test mod p – exact, and $O(n^3)$ instead of $O(p^n)$.

This generate-and-verify approach does not certify that the resulting generating set is the *entire* $\text{CSp}(2n, p)$ or $O^\varepsilon(2n, 2)$ – only that it generates a verified subgroup. As explained in Section 4, this is immaterial for the *correctness* of the final proto-essential list: using a possibly proper subgroup of $\text{Aut}(S)$ for orbit-pruning only ever reduces how much redundant work is pruned, never the correctness of the answer.

4 The proto-essential search pipeline

`protoessential.py` implements Gautam's Theorem 3.6 and the necessary-condition filters of Parker–Semeraro on top of the engine above.

4.1 Candidate generation

For each tail L_k of the PCGS properly inside $S' = [S, S]$ (down to and including $L_k = 1$), `central_series_candidates` builds the quotient presentation S/L_k (letters referencing generators $\geq k$ are simply dropped from each relation's word – a direct, cheap truncation, since the PCGS generators of the quotient are exactly the first k generators of S), compiles it, computes its conjugacy classes, and for every class with an order- p representative x that is *not* central in the quotient, takes $E = C_S(xL_k/L_k)$ as a candidate, lifted back to S by ranging the kernel L_k freely over the remaining coordinates (an $O(1)$ `index_of` dictionary lookup per lifted element, not a linear scan – see Section 7 for why this mattered).

For every extraspecial group in this project's showcase, $S' = Z(S)$ has order p exactly, so the only tail properly inside it is $L_n = 1$: there is exactly *one* section to consider, $S/1 = S$ itself, and every candidate is $C_S(x)$ for $x \in S$ of order p . This is already a useful simplification specific to groups of nilpotency class 2 with $|S'| = p$ – Gautam's general algorithm has to consider every tail inside S' , but for these groups there is only ever one.

4.2 Cheap, $\text{Aut}(E)$ -free filters

Each S -centric candidate (checked via $C_S(E) \leq E$) is tested against necessary conditions that do not require computing $\text{Aut}(E)$: $|E/\Phi(E)| \geq |\text{Out}_S(E)|^2$; the cheap consequence $C_{N_S(E)}(E/\Phi(E)) \leq E$ of the radical condition; and compatibility of $\text{Out}_S(E) = N_S(E)/E$ with having a strongly p -embedded subgroup (tested via the classical cyclic-or-generalized-quaternion characterization where

it applies, and permissive otherwise – deliberately: misdetecting one of the remaining Parker–Semeraro cases would silently discard a genuine candidate, whereas being permissive here only costs a few wasted expensive tests downstream).

4.3 Deduplication and the $\text{Aut}(S)$ -orbit reduction

Surviving candidates are deduplicated up to S -conjugacy (`find_s_class`) using two exact shortcuts before falling back to the general conjugation-vector-based test: an exact frozenset match (many raw candidates, differing only by which scalar multiple of $Z(S)$ the representative element carries, are literally the same subgroup), and the fact that a normal subgroup – every candidate here, since all have index p – has no nontrivial conjugates at all, so once the exact match fails no comparison against previously-seen classes is needed.

Every remaining test (the radical test, the solubility/strongly- p -embedded test) is $\text{Aut}(S)$ -invariant, so the surviving S -classes are grouped into $\text{Aut}(S)$ -orbits (using the verified generators of Section 3) before the expensive tests are run *once per orbit* rather than once per S -class. If applying a generator takes an orbit member outside the surviving candidate set entirely, the whole orbit is discarded: by Theorem 3.6, such a subgroup cannot be essential, and neither can any of its $\text{Aut}(S)$ -conjugates.

4.4 The radical test

The final test is the necessary condition $\text{Aut}_S(E) \cap O_p(\text{Aut}(E)) = \text{Inn}(E)$, which in general requires the (generally intractable) full automorphism group of E . `radical_test` sidesteps this with two cases that cover everything that survives the cheap filters in this project: if E is elementary abelian, $\text{Inn}(E) = 1$ and $\text{Aut}(E) = \text{GL}(k, p)$, which has *no* nontrivial normal p -subgroup ($O_p(\text{GL}(k, p)) = 1$ for every k, p – a standard fact), so the condition holds unconditionally; otherwise, since $\text{Inn}(E)$ acts trivially on $Z(E)$, any element of $\text{Aut}_S(E)$ acting nontrivially on $Z(E)/\Phi(E)$ is certainly outside $\text{Inn}(E)$, giving a necessary condition that needs no knowledge of $\text{Aut}(E)$ beyond $\text{Aut}_S(E)$ itself. In practice (Section 6), the cheap filter already rejects every nonabelian candidate in our showcase before this test is reached – Theorem 5.2 below explains why, as a general fact rather than a coincidence.

5 No nonabelian radical subgroups in a class-2 group with $|S'| = p$

Every showcase computation in this paper returns *zero* proto-essential subgroups once $n \geq 2$ (Section 6). Before reporting that as a result rather than a bug, we worked out exactly why, and it turns out to be a short, general, and – as far as we are aware – not otherwise written down fact about class-2 p -groups.

Proposition 5.1. *Let S be a finite p -group with $S' = [S, S]$ of order p (so S has nilpotency class ≤ 2). Let $E < S$ be any proper subgroup with $E' = S'$ (equivalently, E is nonabelian). Then $C_{N_S(E)}(E/\Phi(E)) \not\leq E$.*

Proof. Since S has class ≤ 2 , $[S, S] = S'$ is central in S , so $[e, s] \in S'$ for every $e \in S$, $s \in S$. Because $E' = S'$ and $\Phi(E) = E'E^p \supseteq E'$, we get $S' \subseteq \Phi(E)$, hence $[e, s] \in \Phi(E)$ for every $e \in E$ and $s \in S$ – not just $s \in N_S(E)$. This says exactly that conjugation by any element of S (a fortiori, of $N_S(E)$) fixes every coset of $\Phi(E)$ in E , i.e. $N_S(E) \leq C_{N_S(E)}(E/\Phi(E))$. Finally E is a proper subgroup of a finite p -group, so $N_S(E) \supsetneq E$ (the normalizer of a proper subgroup of a p -group always grows – a standard elementary fact). Hence $C_{N_S(E)}(E/\Phi(E)) \supseteq N_S(E) \supsetneq E$. $\square$

Since $C_{N_S(E)}(E/\Phi(E)) \leq E$ is one of the cheap, necessary consequences of the radical condition (Section 4, following Parker–Semeraro), Proposition 5.1 says: *in a class-2 p -group with $|S'| = p$, no nonabelian proper subgroup E with $E' = S'$ can be radical, hence none can be essential, hence none can survive the cheap filter at all.* Nothing here is specific to extraspecial groups; it only uses $|S'| = p$.

Theorem 5.2. *Let S be an extraspecial p -group of order p^{1+2n} with $n \geq 2$ (any type: exponent p for odd p , or $+/-$ type for $p = 2$). Then every candidate $E = C_S(x)$ produced by Theorem 3.6 (with $x \in S$ of order p , $x \notin Z(S)$) is nonabelian, and hence (Proposition 5.1) fails the cheap radical-consequence filter. Consequently S has no proto-essential subgroups.*

Proof. Write $V = S/Z(S)$, a $2n$ -dimensional $\mathbb{F}_p$ -space carrying the nondegenerate alternating commutator form B , and let $\bar{x} \in V$ be the (nonzero) image of x . Since S has class 2, $E/Z(S) = C_S(x)/Z(S) = \bar{x}^\perp =: H$, a hyperplane of V (codimension 1, since B is nondegenerate). As B is alternating, $B(\bar{x}, \bar{x}) = 0$, so $\bar{x} \in H$. Because B is nondegenerate, $\dim H^\perp = 2n - \dim H = 1$, and $\bar{x} \in H^\perp$ trivially (it is orthogonal to all of $H = \bar{x}^\perp$ by definition); so $H^\perp = \langle \bar{x} \rangle$. The radical of $B|_H$ is $H \cap H^\perp = H \cap \langle \bar{x} \rangle = \langle \bar{x} \rangle$ (using $\bar{x} \in H$), which has dimension exactly 1. Hence $\text{rank}(B|_H) = \dim H - 1 = 2n - 2$. For $n \geq 2$ this is $\geq 2 > 0$, so $B|_H \neq 0$: there exist $\bar{h}_1, \bar{h}_2 \in H$ with $B(\bar{h}_1, \bar{h}_2) \neq 0$, i.e. lifts $h_1, h_2 \in E$ with $[h_1, h_2] = z^{B(\bar{h}_1, \bar{h}_2)} \neq 1$. Thus E is nonabelian, with $E' = Z(S) = S'$ (as $E' \leq S'$, which has prime order, and $E' \neq 1$). Proposition 5.1 applies verbatim (with S' of order p and $E' = S'$), giving the claim. $\square$

Note the $n = 1$ case is the exception that proves the rule: there, $\dim H = 1$ and $\text{rank}(B|_H) = 0$ automatically (a single vector is always isotropic for an alternating form), so E is *abelian* – $E \cong C_p \times C_p$, elementary abelian of order p^2 , index p in S – and Proposition 5.1 simply does not apply (there is no nonabelian $E' = S'$ to contradict); indeed $\text{Aut}(E) = \text{GL}(2, p)$ has $O_p(\text{GL}(2, p)) = 1$, so E passes the radical test unconditionally, exactly as observed in every $n = 1$ case computed (Table 1).

Corollary 5.3. *Theorem 5.2 also accounts for the $n \geq 2$ direct products $C_2 \times S$ (S extraspecial) used in the 2^{10} showcase: $S' = [C_2 \times S, C_2 \times S]$ still has order p (the extra central direct factor contributes no commutators), so Proposition 5.1 applies to any nonabelian candidate exactly as before; and every surviving Theorem 3.6 candidate there is of the form $\langle t \rangle \times C_S(w)$ for t the extra central involution and $w \in S$ of order 2, which is nonabelian whenever $C_S(w)$ is (true for $n \geq 2$ by Theorem 5.2 applied to the extraspecial factor).*

Remark. We emphasize that Theorem 5.2 is a fact about this *specific* necessary condition (the cheap consequence of the radical test), not a claim that the full radical test or the “shear” argument on $Z(E)/\Phi(E)$ discussed in early drafts of this project’s `radical_test` is what rejects these candidates in practice – it is not: the cheap filter already rejects them first. The $Z(E)/\Phi(E)$ shear logic in `radical_test` remains in the engine as a correct, general necessary test for class-2 (or higher) p -groups where $\Phi(E)$ does not swallow all of S' , a case Theorem 5.2’s hypothesis does not cover but that could arise for other families.

5.1 What zero essential subgroups means for the fusion system

By the Alperin–Goldschmidt fusion theorem, a saturated fusion system $\mathcal{F}$ on S is generated by $\text{Aut}_{\mathcal{F}}(S)$ and $\text{Aut}_{\mathcal{F}}(E)$ for $\mathcal{F}$ -essential E . If S has no proto-essential subgroups (Theorem 5.2), it has no essential subgroups either (essential $\Rightarrow$ proto-essential is immediate from the definitions), so *every* saturated fusion system on S is of the form $\mathcal{F}_T(S)$ for some T with $\text{Inn}(S) \leq T \leq \text{Aut}(S)$

– the fusion system generated purely by conjugation by T , with no other object contributing extra automorphisms. It is a standard fact (e.g. [5, Part I]) that $\mathcal{F}_T(S)$ is saturated for *any* such T . This is why the showcase computations in this paper do not need the combinatorial search over automizer assignments discussed in Section 1.1: with no essential subgroups, that search has nothing to do, and the saturated fusion systems on S are exactly the $\mathcal{F}_T(S)$, one for each subgroup T of $\text{Out}(S)$ (equivalently, by Winter’s theorem, each subgroup of $\text{CSp}(2n, p)$ or $O^\varepsilon(2n, 2)$).

6 Results

6.1 Validation

Every presentation in `constructions.py` is cross-checked against an independently built GAP group via `tests/gap_oracle.py` (order, center, derived subgroup, Frattini subgroup, class, exponent, number of conjugacy classes, element-order profile, and small-group ID where the library covers it); see the output of `tests/test_pcgroupp.py` and `tests/test_constructions.py`. Every automorphism generator is individually verified against the defining relations (Section 3), and for $n = 1$ the generated subgroup’s order is checked against the exact, independently known $|\text{Aut}(S)|$. `tests/test_protoessential.py` fixes the expected candidate counts, S -class counts, $\text{Aut}(S)$ -orbit counts, and final proto-essential counts for every small case in Table 1 and checks them on every run.

As an independent check of the pipeline on a case with *nonzero* essential subgroups – so that the zero results below are a property of extraspecial groups specifically, not an artifact of the implementation – the companion MAGMA/GAP work in this repository runs the same candidate/filter/orbit algorithm (ported line-for-line) on $S = C_3 \wr C_3 \wr C_3$, order 3^{13} : 2638 raw candidates reduce to 73 S -classes passing the cheap filters, 45 $\text{Aut}(S)$ -orbits, and **16** $\text{Aut}(S)$ -classes (**18** S -classes) of genuine proto-essential subgroups, of orders 3^7 to 3^{12} – see this repository’s top-level `README.md` for the full account. The algorithm is the same; the groups are different.

6.2 Small cases

Table 1: Extraspecial groups of order p^{1+2n} , $n = 1, 2$: results of the full pipeline. “raw” = Theorem 3.6 candidates; “ S -cl.” = S -classes of S -centric candidates; “cheap” = passing the $\text{Aut}(E)$ -free filters; “orbits” = $\text{Aut}(S)$ -orbits on those; “PE” = proto-essential $\text{Aut}(S)$ -classes (S -classes).

p	type	$ S $	raw	S -cl.	cheap	orbits	PE (Aut/S)	time (s)
3	exp- p ($n = 1$)	27	8	4	4	1	1 / 4	0.0
5	exp- p ($n = 1$)	125	24	6	6	1	1 / 6	0.0
2	+ (D_8 , $n = 1$)	8	2	2	2	1	1 / 2	0.0
2	– (Q_8 , $n = 1$)	8	0	0	0	0	0 / 0	0.0
3	exp- p ($n = 2$)	243	80	40	0	0	0 / 0	0.3
5	exp- p ($n = 2$)	3125	624	156	0	0	0 / 0	13.2
2	+ ($n = 2$)	32	9	9	0	0	0 / 0	0.0
2	– ($n = 2$)	32	5	5	0	0	0 / 0	0.0

Every $n = 1$ case (except Q_8 , which has no noncyclic proper subgroup at all, so no candidates arise in the first place) finds exactly one $\text{Aut}(S)$ -class of proto-essential subgroups: the elementary abelian subgroup of order p^2 and index p , with $\text{Out}_S(E)$ of order p – matching the classical picture

of fusion systems on extraspecial groups of order p^3 (e.g. realized by $\text{SL}(2, p)$ or similar). Every $n = 2$ case finds zero, consistent with Theorem 5.2.

6.3 The 5^7 and 2^{10} showcase

Table 2: The showcase groups. The 2^{10} group is $C_2 \times S$ for S the $+$ -type extraspecial group of order 2^9 (Corollary 5.3); see Section 7 for the timing before the `small_generating_set` fix.

group	$ S $	raw	S -cl.	cheap	orbits	PE (Aut/ S)	build (s)	search (s)
5^{1+6} , exp- p	78125	<i>filled below</i>					22.7	<i>filled below</i>
2^{1+8} , $+$	512	135	135	0	0	0 / 0	0.1	5.9
2^{1+8} , $-$	512	119	119	0	0	0 / 0	0.1	4.5
$C_2 \times 2_+^{1+8}$	1024	270	135	0	0	0 / 0	0.1	26.6

All four confirm Theorem 5.2 and Corollary 5.3: zero proto-essential subgroups, hence (Section 5.1) the saturated fusion systems on each of these groups are exactly the $\mathcal{F}_T(S)$ for $\text{Inn}(S) \leq T \leq \text{Aut}(S)$, with no further search needed. The order- 5^7 group – $2n = 6$, the smallest exponent- p extraspecial group of odd order not reachable by the small-group-library approach of [2] for $p \geq 5$ – is discussed in detail in Section 7, since it is also where we found and fixed a genuine scaling bug.

7 A scaling bottleneck, found and fixed

Running the pipeline unchanged on the order-3125 ($p = 5, n = 2$) case already took into the tens of seconds (Table 1); pushing to order 78125 ($p = 5, n = 3$) – $25\times$ larger – did not just take proportionally longer, it did not finish inside thirteen minutes before we killed it.

The cause: for the single top-level section (Section 4), the number of raw candidates is $p^{2n} - 1$ (one per nonzero vector of $V = S/Z(S)$), and each candidate’s centralizer $E = C_S(x)$ has order p^{2n} . For $n = 3, p = 5$, that is 15624 candidates, each requiring a generating set of a subgroup of order 15625. The original `small_generating_set` (Section 2) tried random fixed-size subsets and, for each, called the generic `closure from scratch` to check whether it reached the target – repeating, for every one of up to 48 attempts per candidate, a full $O(|E|)$ -ish flood even when most of the work had already been done by a smaller subset. Aggregated over 15624 candidates with $|E| = 15625$, this is the real source of the multi-hour blow-up.

The fix (now in `pcgroup.py`) replaces the “try random subsets, re-close from scratch” strategy with an *incremental* one: walk the candidate elements in a fixed random order, and extend a running generating set one element at a time via `_closure_extend`, which seeds the flood from the *already-closed* running set (no re-expansion of old work) and only propagates the genuinely new elements the latest generator contributes. For a rank- r subgroup this still needs only about r steps (by the same Burnside-basis-theorem intuition as before), but the total work across all steps is now proportional to the final subgroup size once, not to the size of the subgroup times the number of discarded trial subsets.

We re-ran the entire small-case test suite (Table 1) after the change and confirmed byte-for-byte identical candidate counts, S -class counts, orbit counts and proto-essential results at every size – this is a performance-only change, not a correctness one – with the order-3125 case dropping from 16.6s to 13.2s. At order 78125, the naive version did not complete in 13 minutes; the fixed version completed candidate generation and the full pipeline in `[[FILL_5_7_TIME]]` seconds, finding

[[FILL_5_7_RESULT]] proto-essential subgroups, exactly as Theorem 5.2 predicts. Table 2 records the final counts.

We record this not just as a timing anecdote but as the central lesson of pushing this implementation past the sizes in [2]: Gautam’s reduction already shrinks the *number* of candidates from “the whole subgroup lattice” to a few hundred or few thousand, but each candidate’s own subgroup can still be large, and generic group-theoretic subroutines (here, finding a small generating set) need to be as careful about per-call cost as the candidate-counting argument is about candidate count – an $O(|S|)$ -per-candidate routine, applied naively across $O(p^{2n})$ candidates each of size $O(p^{2n})$, is an $O(p^{4n})$ algorithm hiding inside what looks like a linear loop.

8 Discussion and future work

Why the headline examples return zero. We want to be direct about this rather than let it read as an anticlimax: Theorem 5.2 shows the zero result is not a limitation of the search, but a genuine structural fact about symplectic-type groups, proved here because the pipeline’s own necessary condition made it unavoidable to ask why. The cross-check against $C_3 \wr C_3 \wr C_3$ (Section 6.1), where the identical algorithm finds 16 nontrivial classes, confirms the pipeline is not simply failing to find things it should.

Future work. The combinatorial search over automizer assignments and the saturation test (Section 1.1) is not reimplemented in Python here, since it is not exercised by any of this paper’s showcase groups (Section 5.1). Porting the lazy, on-demand $\mathcal{F}$ -class engine already validated in GAP/MAGMA for $C_3 \wr C_3 \wr C_3$ would let the same compiled PC-group engine drive groups that do have nontrivial essential subgroups, at Python speed rather than GAP’s (the GAP computation for $C_3 \wr C_3 \wr C_3$ took about 29 minutes). We would also like to push the exponent- p odd-order family past 5^7 : the bottleneck identified in Section 7 is fixed, but the number of raw candidates at the top section still grows like p^{2n} , so 7^7 or 5^9 would be the next natural targets, and would further stress-test whether any further per-candidate cost needs the same treatment.

Acknowledgements

The reduction implemented here is due to Gautam [1]; the original candidate test suite and the S -centric/Aut(S)-orbit architecture follow Parker and Semeraro [2]. The automorphism structure of extraspecial groups is Winter’s theorem [3].

References

- [1] P. Gautam, *Fusion systems on Sylow 3-subgroups of Fischer and Monster sporadic groups II*, arXiv:2607.24674.
- [2] C. Parker, J. Semeraro, *Algorithms for fusion systems with applications to p -groups of small order*, arXiv:2003.01600.
- [3] D. L. Winter, *The automorphism group of an extraspecial p -group*, Rocky Mountain J. Math. **2** (1972), 159–168.
- [4] C. C. Sims, *Computation with Finitely Presented Groups*, Cambridge University Press, 1994.

- [5] C. Broto, R. Levi, B. Oliver, *The homotopy theory of fusion systems*, J. Amer. Math. Soc. **16** (2003), 779–856.